Demonstration: an API, a web page, and a PDF

Rohan Alexander

16 October, 2026

Introduction

We gather data three ways, from an API, from a web page, and from a PDF. These are in decreasing order of how much the owner of the data wants us to have them. The workflow is the same each time, and it is the one from Week 1:

1. Plan and simulate what we expect to end up with.
2. Get the raw material once, politely, and save it exactly as it arrived.
3. Read from disk, and turn it into rows and columns.
4. Test it.
5. Save Parquet.

As in earlier weeks Python (Astral 2025; Vink et al. 2025) does this, and R (R Core Team 2025; Richardson et al. 2025; Wickham 2016; Arel-Bundock 2025) reads the Parquet files to make graphs and tables. The scripts are in `scripts/`. Set up the environment and run them with:

```
uv sync
uv run scripts/01-api.py
uv run scripts/02-scrape.py
uv run scripts/00-simulate.py
uv run scripts/03-parse_pdf.py
```

```
library(arrow)
library(dplyr)
library(ggplot2)
library(tinytable)
```

An API

Open Data Toronto does not need a key. Most APIs do, so the script shows the pattern anyway. The key goes in a file called `.env`, which looks like this, and which is listed in `.gitignore` before it is created:

```
EXAMPLE_API_KEY=abc123
```

The code then only ever refers to the key by name.

```
import os
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.environ.get("EXAMPLE_API_KEY")
```

On GitHub Actions the same name is set as a repository secret, as in the Week 1 assignment, and the same code works.

The request is a base URL, an endpoint, and parameters. We identify ourselves in the header so that if we cause a problem then the people who run the server know who to email.

```
BASE_URL = "https://ckan0.cf.opendata.inter.prod-toronto.ca/api/3/action"
HEADERS = {"User-Agent": "INF3104 class demonstration (rohan.alexander@utoronto.ca)"}

response = requests.get(
    f"{BASE_URL}/package_show",
    params={"id": "daily-shelter-overnight-service-occupancy-capacity"},
    headers=HEADERS,
    timeout=30,
)
response.raise_for_status()
package = response.json()

# Save exactly what came back before doing anything to it
with open("data/raw/package_show.json", "w") as f:
    json.dump(package, f, indent=2)
```

What comes back is nested. We decide that a row is a file, and pull out the fields that we want from each.

Table 1: The files that make up the shelter occupancy dataset, according to the Open Data Toronto API

format	in_datastore	n
CSV	FALSE	6
CSV	TRUE	6
JSON	FALSE	6
XML	FALSE	6

```
resources = pl.DataFrame(
    [
        {
            "name": r["name"],
            "format": r["format"],
            "in_datastore": r["datastore_active"],
            "last_modified": r["last_modified"],
            "id": r["id"],
        }
        for r in package["result"]["resources"]
    ]
)
```

```
resources <- read_parquet("data/resources.parquet")

resources |>
  count(format, in_datastore) |>
  tt() |>
  format_tt(escape = TRUE)
```

The script then pauses for a second, and asks the `datastore_search` endpoint for the 500 most recent rows of the current file. Look at what type everything is.

```
latest <- read_parquet("data/shelter_latest_500.parquet")

latest |>
  select(OCCUPANCY_DATE, SECTOR, SERVICE_USER_COUNT, CAPACITY_ACTUAL_BED) |>
  glimpse()
```

```
Rows: 500
```

```
Columns: 4
$ OCCUPANCY_DATE      <chr> "2026-09-16", "2026-09-16", "2026-09-16", "2026-
09~
$ SECTOR              <chr> "Youth", "Youth", "Youth", "Youth", "Youth", "Wome~
$ SERVICE_USER_COUNT  <chr> "31", "8", "20", "32", "30", "43", "31", "31", "25~
$ CAPACITY_ACTUAL_BED <chr> "41", "8", "20", "33", "30", "43", "31", "31", "25~
```

```
latest |>
  count(CAPACITY_TYPE, bed_capacity_is_missing = is.na(CAPACITY_ACTUAL_BED))
```

```
# A tibble: 2 x 3
  CAPACITY_TYPE       bed_capacity_is_missing     n
  <chr>               <lgl>                   <int>
1 Bed Based Capacity  FALSE                     372
2 Room Based Capacity TRUE                      128
```

Everything is a character, including the counts. And the number of beds is missing for about a quarter of the rows. This is because those programs are measured in rooms rather than because someone forgot. An API gives us the data. It does not give us a dataset. Turning one into the other is next week.

A web page

Wikipedia has an API, and for real work we would use that. We use the page listing the prime ministers of Canada because everyone can look at it, and because it is a realistic mess, with merged cells, footnotes, and prime ministers who served more than once.

We get the page once and save it. Everything after that reads from disk, so we do not go back to the website every time we fix our code, and we will need to fix our code.

```
RAW = Path("data/raw/prime_ministers.html")

if not RAW.exists():
    response = requests.get(URL, headers=HEADERS, timeout=30)
    response.raise_for_status()
    RAW.write_text(response.text, encoding="utf-8")

soup = BeautifulSoup(RAW.read_text(encoding="utf-8"), "html.parser")
```

Using “Inspect” in the browser, each name is a bold link in a table with class `wikitable`, followed by the years they were alive. So we find that, and use a regular expression for the years.

It took three attempts, which is normal, and each failure is instructive.

1. The first table on the page with class `wikitable` is the key to the abbreviations for the provinces, and so the first attempt returned nothing at all. We want the table with the most rows.
2. The second attempt returned 17 prime ministers. The test that there should be at least 23 failed. The pattern was looking for `(1815-1891)` and living prime ministers are written as `(b. 1939)`.
3. One cell has spaces inside the parentheses. That did not cost us a row, but only because that prime minister served twice and so appears twice.

Without the test the second attempt would have looked fine. It was a clean table of 17 rows, with no error and no warning. It would also have been missing, systematically, every prime minister who is still alive.

```
tables = soup.find_all("table", class_="wikitable")
table = max(tables, key=lambda t: len(t.find_all("tr")))

rows = []
for cell in table.find_all(["td", "th"]):
    bold_link = cell.select_one("b > a")
    span = re.search(
        r"\((?:b\.\s*)?(\d{4})(?:\s*[--]\s*(\d{4}))?\s*\)", cell.get_text(" ")
    )
    if bold_link and span:
        rows.append(
            {
                "name": bold_link.get_text(strip=True),
                "born": int(span.group(1)),
                "died": int(span.group(2)) if span.group(2) else None,
            }
        )

prime_ministers = pl.DataFrame(rows).unique(subset="name", maintain_order=True)

assert prime_ministers.height >= 23, "Fewer prime ministers than there have been"
assert prime_ministers["name"][0] == "John A. Macdonald"
assert prime_ministers["born"].is_between(1800, 2000).all()
```

```
prime_ministers <- read_parquet("data/prime_ministers.parquet")

prime_ministers |>
  mutate(
    alive = is.na(died),
    end = if_else(alive, 2026L, died),
    name = factor(name, levels = rev(name))
  ) |>
  ggplot(aes(y = name, colour = alive)) +
  geom_linerange(aes(xmin = born, xmax = end)) +
  scale_colour_manual(values = c("black", "grey60"), guide = "none") +
  labs(x = "Year", y = NULL) +
  theme_minimal()
```

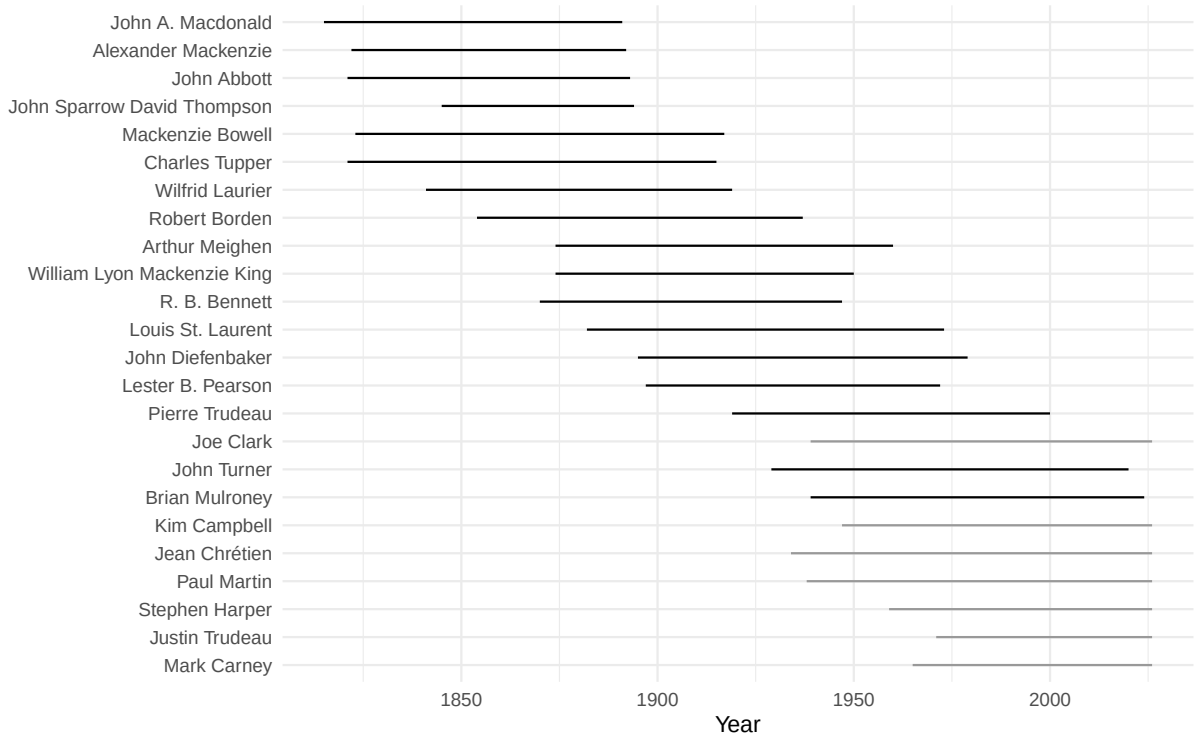


Figure 1: How long each Canadian prime minister lived, from the scraped Wikipedia table. Those still alive are in grey and are shown to 2026.

A PDF

The Dysart paper asks you to turn a table in a DHS final report from the 1980s or 1990s into a dataset. Here we do that for one small table, Table 3.1 of the final report of the 1993 Kenya DHS, which has age-specific fertility rates by urban and rural residence. (As it has now been done, it is not one that you can use.)

We first simulate what we expect, which is seven age groups, three residence categories, and a rate that is a whole number of births per 1,000 women, somewhere between 0 and 400. That gives us 21 rows and three tests before we have opened the PDF.

The PDF goes into `inputs/`. This report is a scan to which OCR has already been applied, so there is a text layer and we can extract it without running OCR ourselves. We save that first parse before we touch it. After that the script reads from the saved file, as it did with the web page.

```python
FIRST_PARSE = Path("data/raw/first_parse.txt")
if not FIRST_PARSE.exists():
    with pdfplumber.open(PDF) as pdf:
        FIRST_PARSE.write_text(pdf.pages[PAGE - 1].extract_text())

# Everything after this reads from the saved first parse
text = FIRST_PARSE.read_text()
```

```r
readLines("data/raw/first_parse.txt", warn = FALSE)[6:22] |>
  writeLines()
```

```
Age group Urban Rural Total
15-19 74 119 110
20-24 176 281 257
25-29 178 258 241
30-34 133 209 197
35-39 74 166 154
40-44 (34) 74 70
45-49 (21) 53 50
TFR 15-49 3.44 5.80 5.40
TFR 15-44 3.34 5.54 5.15
GFR 129 194 182
CBR 35.1 35.9 35.8
Note: Rates axe for the period 1-36 months
preceding the survey. Rates for age group
45-49 may be slightly biased due to
```

```
truncation. Rates in parentheses are based
on fewer than 250 woman-yeaxs of
```

The table has survived. The note under it has not, and it now says “Rates axe” and “woman-yeaxs”. If the OCR can turn “r” into “x” in the note then it can turn “3” into “8” in the table, and we would not see it by looking.

We rebuild the table with a regular expression. A row that we want is an age group and then three numbers, some in parentheses. The note says that parentheses mean that the rate is based on fewer than 250 woman-years of exposure. That is information about the quality of the measurement, and so it gets its own column. We do not throw it away and we do not leave it in the number.

Then the tests. The first three came from the simulation. The last is a gift from the table. The total fertility rate is five times the sum of the age-specific rates, divided by 1,000, and the table reports it. So the table contains its own check. Most tables in reports do, in the form of totals. Use them.

```python
reported_tfr = {"Urban": 3.44, "Rural": 5.80, "Total": 5.40}
for residence, reported in reported_tfr.items():
    rates = fertility.filter(pl.col("residence") == residence)["births_per_1000_women"]
    implied = 5 * rates.sum() / 1000
    assert abs(implied - reported) < 0.02, (residence, implied, reported)
```

```r
fertility <- read_parquet("data/kenya_1993_fertility.parquet")

fertility |>
  filter(residence != "Total") |>
  ggplot(aes(x = age_group, y = births_per_1000_women, group = residence)) +
  geom_line(aes(linetype = residence)) +
  geom_point(aes(shape = small_exposure), fill = "white", size = 2) +
  scale_shape_manual(values = c(19, 21), guide = "none") +
  labs(x = "Age group", y = "Births per 1,000 women", linetype = "Residence") +
  theme_minimal()
```

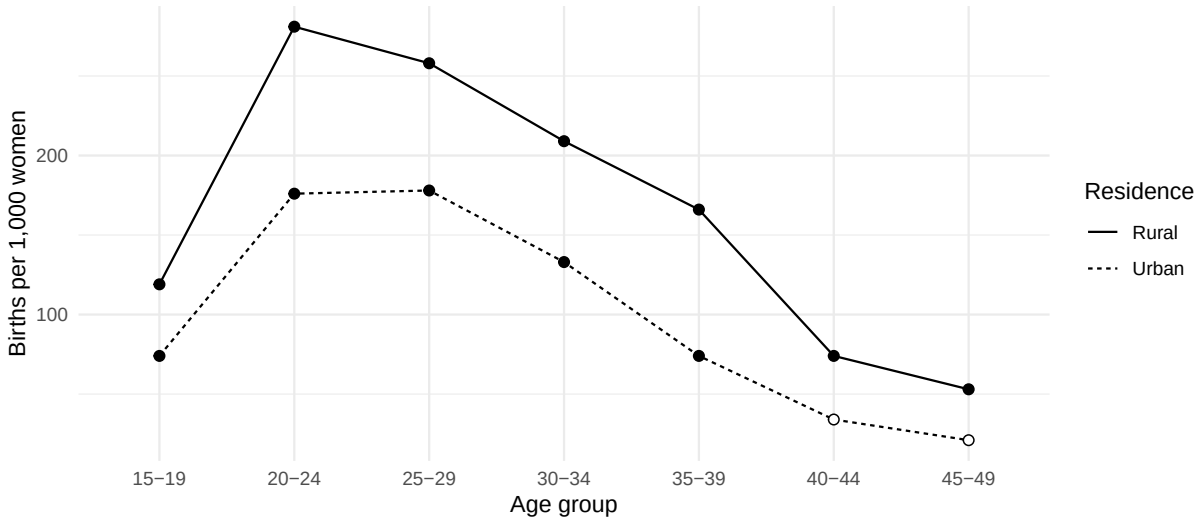


Figure 2: Age-specific fertility rates in the three years before the 1993 Kenya DHS, by residence. Hollow points are based on fewer than 250 woman-years of exposure.

Figure 2 is the figure on the same page of the report, remade from the data that we recovered. Compare the two.

References

Arel-Bundock, Vincent. 2025. *tinytable: Simple and Configurable Tables in HTML, LaTeX, Markdown, Word, PNG, PDF, and Typst Formats*. https://vincentarelbundock.github.io/tinytable/.

Astral. 2025. *uv: An Extremely Fast Python Package and Project Manager*. https://docs.astral.sh/uv/.

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. https://www.R-project.org/.

Richardson, Neal, Ian Cook, Nic Crane, et al. 2025. *arrow: Integration to Apache Arrow*. https://arrow.apache.org/docs/r/.

Vink, Ritchie et al. 2025. *Polars: DataFrames for the New Era*. https://pola.rs.

Wickham, Hadley. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org.